

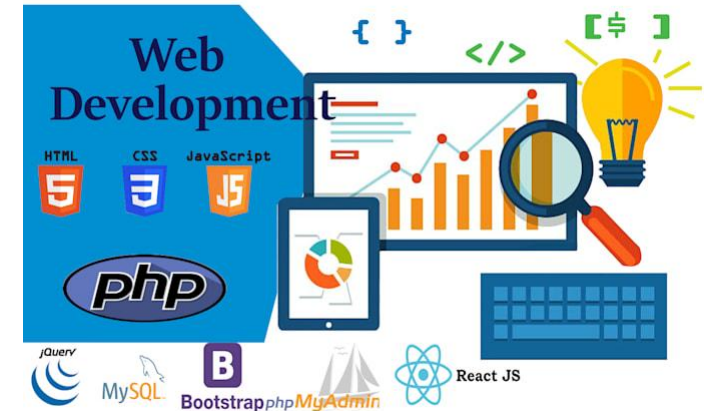
Semestre: 2026.1

Disciplina:

**ARA0062 DESENV. WEB EM HTML5, CSS,
JAVASCRIPT E PHP**

Aula 10

Fortaleza, CE: 12 de maio, 2026



- 1.Entrada e Saída de Dados
- 2.Condicionais (if/else)
- 3.Loops (for, while)
- 4.Arrays
- 5.Funções ← Vamos desenvolver este
tópico
- 6.Strings
- 7.Projeto Final





2. Por intermédio do script externo com extensão **.js** no body

Nome do programa:

script.js

*alert("MLK de programação");
console.log("Era da imaginação");*

```
<!DOCTYPE html>
<html>
<head>
    <title>Exemplo</title>
</head>
<body>
    <h1>Meu Site</h1>
    <script src="script.js"></script>
</body>
</html>
```

JAVASCRIPT NO NAVEGADOR

Interação

window.alert() ou **alert()**

console.log()

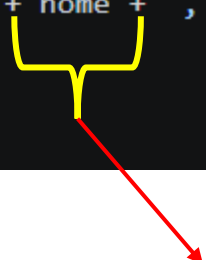
VARIÁVEIS

É um espaço na memória do computador utilizada para guardar um valor. Você dá um nome para esse espaço e depois pode usar esse nome para acessar o valor.

VARIÁVEIS

window.prompt()

```
<script>
  let nome = window.prompt("Qual é o teu nome?")
  window.alert("Olá " + nome + ", prazer em te conhecer e seja bem-vindo ao curso de JavaScript para todos! Aula de hoje é noção sobre variaveis!")
</script>
```



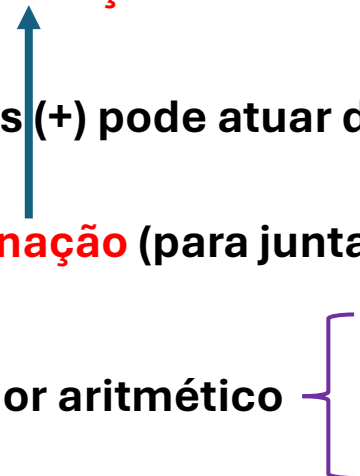
Símbolo de **concatenação**

OBS. De vez em quando sinal mais (+) pode atuar de uma ou duas formas:

1. Como símbolo de **concatenação** (para juntar dois ou mais trechos do código)

2. Como símbolo de operador aritmético

numero1 = 20
numero2 = 10
soma = numero + numero2



Atenção:

Para operações aritméticas, é preciso utilizar a conversão do tipo, visto que, o JS entende dado como String.

VARIÁVEIS

Regras para declarar variável (is):

Pode conter letras, números, _ e \$

Não pode começar com número

Não pode usar palavras reservadas (ex: let, if, for)

JavaScript é case-sensitive (maísculas e minúsculas são diferentes)

```
// Forma correta de declarar variáveis em JavaScript
```

```
let nome = "Rodrigo";
```

```
let _sobrenome = "Silva";
```

```
let $idade = 30;
```

```
// Formas incorretas de declarar variáveis
```

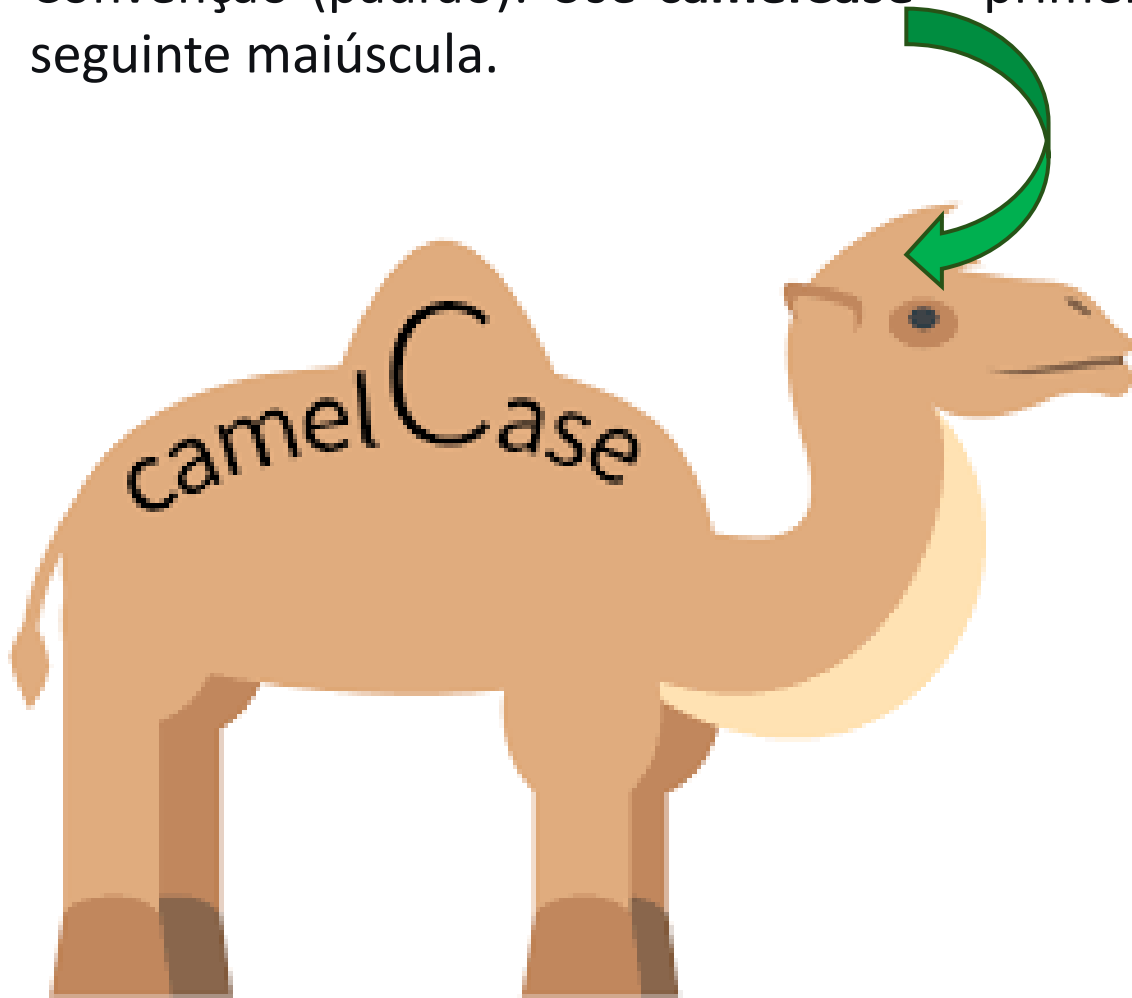
```
let 1nome = "Maria"; // Não pode começar com número
```

```
let sobrenome-completo = "Souza"; // Não pode conter hífen
```

```
let var = "Teste"; // "var" é uma palavra reservada
```

VARIÁVEIS

Convenção (padrão): Use **camelCase** – primeira letra minúscula, primeira letra de cada palavra seguinte maiúscula.



```
// camelCase
```

```
let nomeDoUsuario = "Julia";
```

```
let pontuacaoTotal = 10;
```

```
let estaAprovado = true;
```

Declaração múltipla de variáveis

```
let a = 10, b = 20, c = 30;
console.log(a, b, c);

console.log("=== TESTE 2 ===");
let x, y, z;
x = 5; y = 15; z = 25;
console.log(x + y + z);

console.log("=== TESTE 3 ===");
let nome = "João", idade = 30, aprovado = true;
console.log(nome, idade, aprovado);

console.log("=== TESTE 4 ===");
let p = 10, s = 20, t;
t = p; p = s; s = t;
console.log("p=", p, "s=", s);

console.log("=== TESTE 5 ===");
const PI = 3.14, G = 9.8;
console.log("PI=", PI, "G=", G);

console.log("=== TESTE 6 ===");
let n1 = 15, n2 = 4;
console.log("Soma:", n1 + n2);
console.log("Produto:", n1 * n2);
```

VARIÁVEIS – TIPOS PRIMITIVOS

```
let numero1 = window.prompt("Digite o primeiro número: ")
let numero2 = window.prompt("Digite o segundo número: ")
let resultado = numero1 + numero2
window.alert("O resultado da soma é: " + resultado) // O resultado da soma é: 1020, pois o prompt retorna uma string, e a operação de soma entre strings é a concatenação. Para corrigir isso, precisamos converter as entradas para números usando a função parseFloat() ou parseInt().
```

```
// Para resolver o problema da concatenação, precisamos converter os valores para o tipo numérico (number) usando a função Number() ou parseFloat().
let numero1 = parseFloat(window.prompt("Digite o primeiro número: "))
let numero2 = parseFloat(window.prompt("Digite o segundo número: "))

let resultado = numero1 + numero2

window.alert("O resultado da soma é: " + resultado)
```

VARIÁVEIS – TIPOS PRIMITIVOS

É a classificação que indica qual o formato da informação armazenada.

Tipo	O que guarda	Exemplo	Relação
string	Texto	"João", 'Olá'	Placa de texto
number	Número	25, 3.14, -10	Calculadora
boolean	Verdadeiro/falso	true, false	Interruptor
undefined	Sem valor definido	let x;	Gaveta vazia
null	Vazio intencional	let x = null;	Gaveta com etiqueta "vazia"

VARIÁVEIS – TIPOS PRIMITIVOS

String (texto)

Sequência de caracteres dentro de aspas.

```
let nome = "Ana";  
let sobrenome = 'Silva';  
let frase = `Olá, ${nome}!`; // template string
```

Boolean (booleano)

Apenas dois valores: true (verdadeiro) ou false (falso).

```
let aprovado = true;  
let logado = false;
```

Number (número)

Definição: Valores numéricos (inteiros ou decimais). Sem aspas!

```
let idade = 25;  
let altura = 1.75;  
let temperatura = -5;
```

Undefined

Definição: Variável foi criada mas não recebeu valor.

```
let resposta;  
console.log(resposta); // undefined
```

VARIÁVEIS – TIPOS PRIMITIVOS

Null

Definição: Variável com valor vazio intencional.

```
let resultado = null;
```

Descobrir tipo; função (**typeof**)

```
console.log(typeof "Olá");    // "string"  
console.log(typeof 42);       // "number"  
console.log(typeof true);     // "boolean"  
console.log(typeof undefined); // "undefined"
```

VARIÁVEIS – TIPOS PRIMITIVOS

CONVERSÃO DO TIPO

```
<script>

  let numero1 = window.prompt("Digite o primeiro número: ")
  let numero2 = window.prompt("Digite o segundo número: ")
  let resultado = numero1 + numero2
  window.alert("O resultado da soma é: " + resultado) // O resultado da soma é: 1020, pois o prompt retorna uma string, e a operação de soma
entre strings é a concatenação. Para corrigir isso, precisamos converter as entradas para números usando a função parseFloat() ou parseInt().

</script>
```

Atenção:

Para operações aritméticas, é preciso utilizar a conversão do tipo, visto que, o JS entende dado como String.

parseInt() ou parseFloat()

```
let numero1 = parseInt(window.prompt("Digite o primeiro número: "))
let numero2 = parseInt(window.prompt("Digite o segundo número: "))
// let resultado = window.prompt(numero1 + numero2)
let resposta = window.alert("O resultado da soma é: " + resultado)
```

Formatação de Strings

Processo de montar, organizar e exibir textos utilizando variáveis dentro de uma mensagem. Em JavaScript, isso permite combinar textos fixos com valores armazenados em variáveis.

Template String

É uma forma mais prática e organizada de criar e formatar textos em JavaScript. Permite inserir variáveis dentro de uma string de maneira mais simples e legível. Para o efeito, usa crase (``) dupla e, para inserir variáveis utiliza-se interpolador `${}` de variáveis.

length; toUpperCase(); toLowerCase(); toFixed()

VARIÁVEIS – FORMATAÇÃO DE STRINGS

```
<script>
  let nome = "Rodrigo"
  let curso = "CC"
  let universidade = "Unifanor"
  let disciplina = "Dev Web"
  let nota = 6

  window.alert(`Seu nome é ${nome}, você estuda ${curso}, na ${universidade}, disciplina ${disciplina} e sua nota é ${nota}`) // Seu nome é
  Rodrigo, você estuda CC, na Unifanor, disciplina Dev Web e sua nota é 6

```

```
<script>
  let nome = "Rodrigo"
  let curso = "CC"
  let universidade = "Unifanor"
  let disciplina = "Dev Web"
  let nota = 6

  window.alert("Seu nome é " + nome + ", você estuda " + curso + ", na " + universidade + ", disciplina " + disciplina + " e sua nota é " +
  nota) // Seu nome é Rodrigo, você estuda CC, na Unifanor, disciplina Dev Web e sua nota é 6
</script>

```



VARIÁVEIS – FORMATAÇÃO DE STRINGS

Comando	Nomenclatura
length	Propriedade
toUpperCase()	Método
toLowerCase()	Método
toFixed()	Método

VARIÁVEIS

variavel.toFixed()

```
<script>
  let numero = window.prompt("Digite um número: ") // 3.1415926535897932384626433832795
  console.log(numero.toFixed(2)) // 3.14
  console.log(numero.toFixed(4)) // 3.1416
  console.log(numero.toFixed(10)) // 3.1415926536
</script>
```

```
<script>
  // usando toFixed dentro do documento HTML
  let numero2 = window.prompt("Digite outro número: ") // 3.1415926535897932384626433832795
  document.write(`0 número digitado foi ${numero2} e arredondado para 2 casas decimais fica ${numero2.toFixed(2)} <br>`)
  document.write(`0 número digitado foi ${numero2} e arredondado para 4 casas decimais fica ${numero2.toFixed(4)} <br>`)
  document.write(`0 número digitado foi ${numero2} e arredondado para 10 casas decimais fica ${numero2.toFixed(10)} <br>`)
</script>
```

O que é uma condição?

No sentido coloquial é uma situação ou um requisito que precisa ser atendido para que algo aconteça.

Se tiver cachaça, eu vou para a festa.

E na programação?

Na **programação**, é uma expressão que pode ser avaliada como **verdadeira** ou **falsa** e que determina o fluxo da execução do código.

Condição simples.

Dessa forma, uma **condicional simples** executa um bloco de código apenas **se** uma condição for verdadeira, utilizando **if** (definida como uma estrutura de controle de fluxo)

CONDICIONAIS - SIMPLES

Em linguagem de programação de computadores, por exemplo (JS) :

É a forma mais básica de estrutura condicional em JavaScript. Ela é usada quando queremos executar um bloco de código apenas se uma determinada condição for verdadeira.

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

VEJAMOS O EXEMPLO A SEGUIR:

```
<script>  
    let time = "Fortaleza SC"  
  
    if(time == "Fortaleza SC"){  
        window.alert("Você torce para o Fortaleza!")  
    }  
</script>
```

Desafio 01:

A partir do exemplo anterior, modifique a condição para time “Ceará SC”;

```
<script>  
  let time = "Fortaleza SC"  
  
  if(time == "Fortaleza SC"){  
    window.alert("Você torce para o Fortaleza!")  
  }  
</script>
```

Desafio 02:

Use prompt para interação com usuário:

```
<script>
  // Usando o prompt
  let timeEstimacao = window.prompt("Digite o nome do seu time: ")

  if(timeEstimacao == "Fortaleza SC"){
    window.alert("Você torce para o Fortaleza!")
  }
</script>
```

CONDICIONAIS - COMPOSTAS

É uma estrutura de controle que permite executar diferentes blocos de código com base em múltiplas condições. A sintaxe é:

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```

```
<script>
```

```
let time = window.prompt("Digite o nome do seu time:");
```

```
if(time == "Fortaleza SC"){
```

```
    window.alert("Você torce para o Fortaleza!");
```

```
}else{
```

```
    window.alert("Você torce para o Ceará!");
```

```
}
```

```
</script>
```

CONDICIONAIS – COMPOSTAS COM MÚLTIPLAS ALTERNATIVAS

São estruturas de controle que permitem testar várias condições e executar diferentes blocos de código com base no resultado.

A sintaxe é a seguinte:

```
// if (condição1) {  
//   // código a ser executado se condição1 for verdadeira  
// } else if (condição2) {  
//   // código a ser executado se condição2 for verdadeira  
// } else if (condição3) {  
//   // código a ser executado se condição3 for verdadeira  
// } else {  
//   // código a ser executado se nenhuma das condições anteriores for verdadeira  
// }
```

BOLAR UM PLANO PARA SAIR....

```
<script>  
  
  let time = window.prompt("Digite o nome do seu time:")  
  
  if(time == "Fortaleza SC"){  
    window.alert("Você torce para o Fortaleza!")  
  }else if(time == "Ceará SC"){  
    window.alert("Você torce para o Ceará!")  
  }else if(time == "Flamengo"){  
    window.alert("Você torce para o Flamengo!")  
  }else{  
    window.alert("Time desconhecido!")  
  }  
  
</script>
```



LOADING



Loops em JavaScript

São estruturas de repetição utilizadas para executar um bloco de código várias vezes. Evitam repetição manual de comandos.

LOOPS

O loop **for** é utilizado quando sabemos quantas vezes o código deve repetir.

SINTAXE:

```
for(inicialização; condição; incremento){  
  
    // código  
  
}
```

Sem usar for

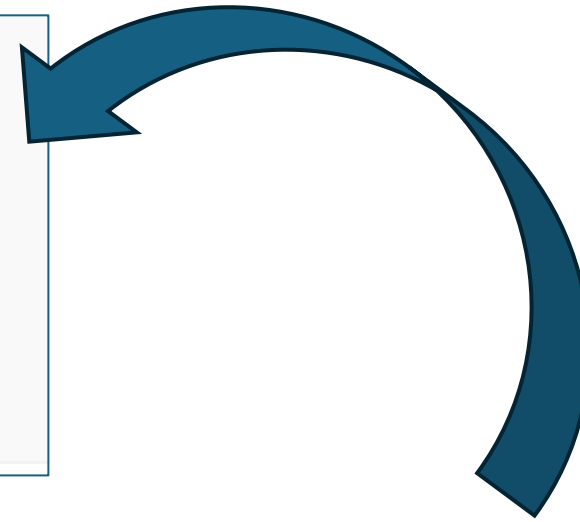
Sem estrutura de repetição, precisaríamos repetir o comando manualmente.

```
<script>  
    // Esta seria a forma de repetir um código 5 vezes sem usar o FOR  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
</script>
```

LOOPS

Com o **for**

```
for(inicialização; condição; incremento){  
  
    // código  
  
}
```



Sem o **for**

```
<script>  
    // Esta seria a forma de repetir um código 5 vezes sem usar o FOR  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
    console.log("Fortaleza")  
</script>
```

Com o **for**

```
<script>  
    // usando o FOR para repetir um código 5 vezes  
    for (let i = 0; i < 5; i++) {  
        console.log("Fortaleza")  
    }  
</script>
```

Loop **while** ou while

Estrutura de controle que permite repetir um bloco de código enquanto uma condição for verdadeira. Ele é útil quando não sabemos exatamente quantas vezes o código precisa ser executado, mas queremos continuar repetindo até que uma condição específica seja atendida.

```
// Sintaxe:  
// while (condição) {  
//     // código a ser executado  
// }
```

Loop **while** ou while

```
// Sintaxe:  
// while (condição) {  
//     // código a ser executado  
// }
```



Loop **while** ou **while**

Laço de repetição que executa um bloco de código enquanto uma condição for verdadeira.

Enquanto o Fortaleza continuar dando moleza, o Ceará vai levando troféus estadual.



Loop **while** ou while

Laço de repetição que executa um bloco de código enquanto uma condição for verdadeira.

Enquanto o Ceará não for para liberta, o Fortaleza vai continuar sendo o time de elite sul-americana.



Loop **while** ou while

```
<script>

let contador = 1

while(contador <= 5){

    window.alert("Fortaleza")

    contador++

}

</script>
```

Desafio com loop **while**

```
// Exemplo: Tabuada do 7 usando while
// console.log("--- Tabuada do 7 ---");
// let n = 1;
// while (n <= 5) {
//   console.log(`7 x ${n} = ${7 * n}`);
//   n++; // Incrementa n para a próxima multiplicação
```